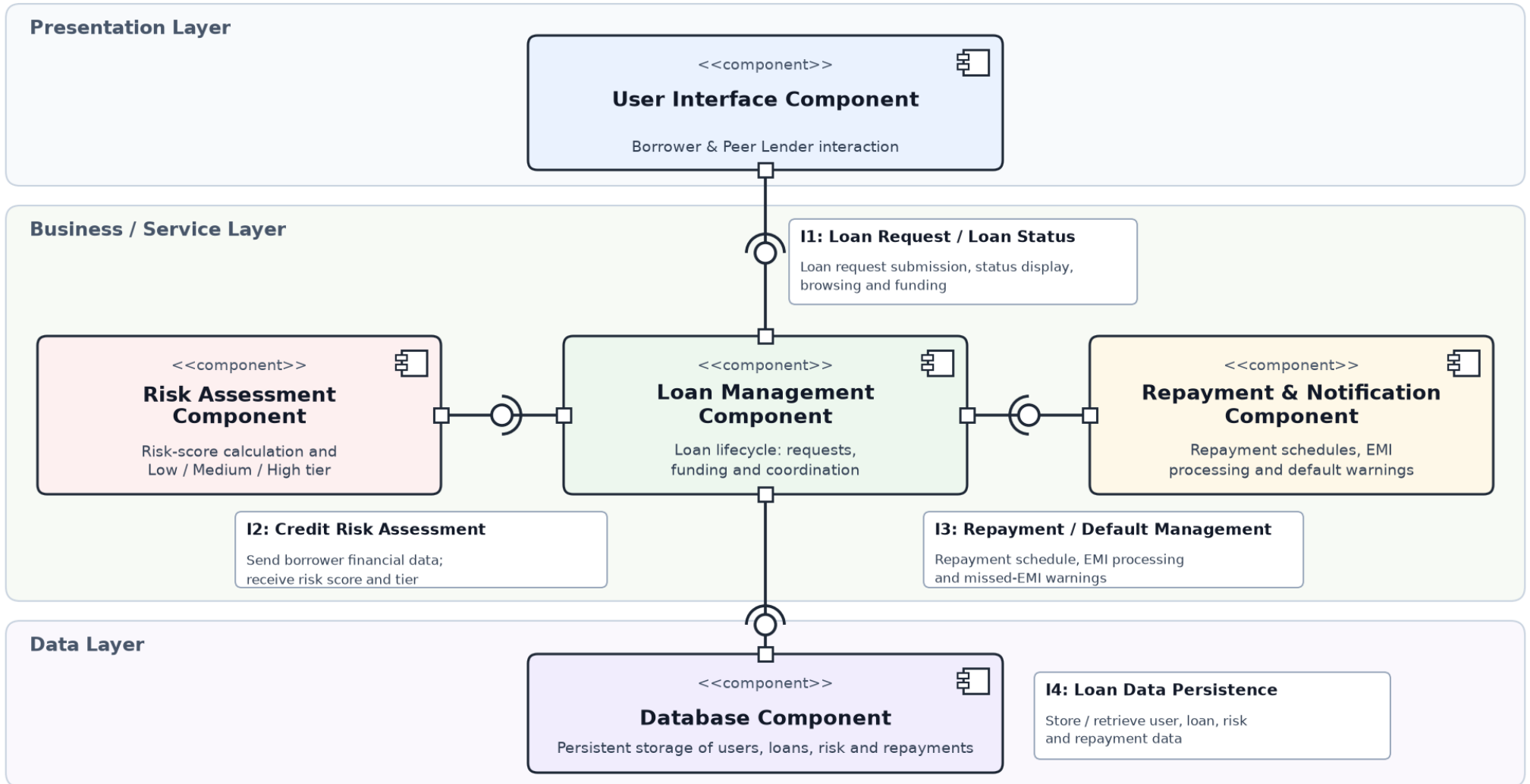


Micro-Lending & Peer Credit Risk Assessor

UML Component Diagram – Layered Architecture



UML Notation

○ Provided interface (ball) ◐ Required interface (socket)

5 components • 4 interfaces • □ = port

Software Engineering Lab 3

Component Modelling & Architectural Pattern Selection

Name: Apurv Kumar Singh SRN: PES1UG24CS077 Class: 5B

Project: Micro-Lending & Peer Credit Risk Assessor

Architecture Selection

We chose Layered Architecture for the Micro-Lending & Peer Credit Risk Assessor. The architecture separates user interaction, core loan processing, risk assessment, repayment/default management, and persistent data storage into clear responsibilities.

Reason 1 - Clear separation of responsibilities

The system contains distinct activities such as loan submission, credit-risk evaluation, funding, repayment, and default management. Separating these responsibilities makes the design easier to understand and allows risk-assessment rules to be modified without redesigning the user interface or persistence logic.

Reason 2 - Suitable complexity and maintainability

The platform has a well-defined lending workflow and does not require independently deployed services for every function. Layered Architecture provides modularity with less operational and network complexity than a microservices design, while keeping deployment and data flow straightforward.

Security Advantage

Sensitive financial operations can be controlled through the business and data layers, with centralized validation and transaction handling before data is committed. This supports the ACID transaction requirement and reduces the risk of inconsistent repayment records or double-spending.

Performance Benefit

Requests can be processed within a single application through well-defined layers, avoiding unnecessary distributed service-to-service network calls. This reduces communication overhead and helps the risk-assessment and repayment dashboard meet the required responsive experience.

Architecture Comparison (Summary)

Layered	Clear separation; simple maintenance; best fit for this system.
Microservices	Independent scaling/fault isolation, but adds operational and network complexity.
Client-Server	Centralized control and deployment, but greater bottleneck/single-failure risk.